

IPv4 vs IPv6

Errata & Further Reading

An optional companion to the slide deck and lab workbook — for anyone who wants to go a level deeper, plus a few precision corrections to claims you'll often hear repeated. Nothing here is required for the lab; it's here so the curious (and the pedantic) leave satisfied.

NOTE: How this is organized — A short history of how we ended up straddling two protocols, two deep dives the workshop only had time to name-drop (IPsec and the death of broadcast), an errata section that sharpens some common-but-imprecise statements, and a one-stop RFC index that matches the timeline diagram. Skim what interests you.

1. How did we get here? A brief history

The workshop treats IPv4 and IPv6 as engineering. The reason we're stuck between them, though, is mostly history and economics — and it's the part most attendees find surprising.

4.3 billion once looked infinite

When IPv4 was specified (RFC 791, 1981), the entire internet was a research network of a few hundred hosts. $2^{32} \approx 4.3$ billion addresses seemed absurdly large, so no one minded that the early “classful” A/B/C scheme wasted enormous blocks (a single Class A handed one organization 16 million addresses). Address frugality simply wasn't a design goal — there was no reason to think it mattered.

The protocol wars — and why TCP/IP won

Through the 1980s and early '90s, TCP/IP was not the obvious winner. It competed with the government-blessed OSI suite and with proprietary stacks: Novell's IPX/SPX, Apple's AppleTalk, Digital's DECnet, and IBM's SNA. TCP/IP won — but notice that address size had nothing to do with it. It won for non-technical reasons:

- Open and vendor-neutral — it belonged to no one, so no competitor had to bet its network on a rival (IPX was Novell's, AppleTalk Apple's, SNA IBM's).
- Free, working code — DARPA funded a TCP/IP stack that shipped in Berkeley Unix (4.2BSD, 1983). Every university and workstation vendor got a working implementation for free.
- “Rough consensus and running code” — TCP/IP was deployed and iterating while OSI, the officially mandated standard, was still being designed by committee. OSI was too complex and too late.
- The web (1991) and the ISPs standardized on it — so the network everyone wanted to join simply was the TCP/IP internet.

Did anyone see exhaustion coming?

Not in 1981. The alarm was raised around 1990–1992, and — tellingly — the first worry wasn't raw exhaustion but Class B exhaustion and the routing table growing faster than routers could cope. The fixes arrived in a tight cluster, which is why NAT can feel like it appeared “right after” IPv4 won:

Year	Milestone
1981	RFC 791 — IPv4, with 32-bit classful addressing
1983	ARPANET cuts over to TCP/IP; 4.2BSD ships a free TCP/IP stack
1990–91	IETF raises the alarm on Class B exhaustion + routing-table growth
1993	CIDR (RFC 1519) — kills classful waste, enables route aggregation
1994	NAT (RFC 1631) and private space (RFC 1597) — the stopgaps
1994–95	IPng effort → IPv6 (RFC 1883) — the long-term fix
1996	RFC 1918 — private address space, finalized

So the gap between “IPv4 wins” and “IPv4 is running out” was small not because the design was rushed, but because the same event — the commercial/web explosion of the early '90s — caused both at once.

Wasn't IPX basically IPv6 already?

There's a real kernel of truth here, plus a common myth to clear up. The myth first: TCP/IP is older than IPX, not newer. TCP/IP dates to the mid-1970s (cut over on ARPANET in 1983); IPX came out of Novell NetWare around 1983–85 and was itself derived from Xerox's XNS from the late 1970s. What feels ancient about IPX is really XNS being a parallel early effort.

The kernel of truth: an IPX address is a 32-bit network number plus a 48-bit node that is just the interface's MAC address (≈80-bit addressing). Hosts auto-configured their node ID from hardware and per-network host space was effectively unlimited — which is strikingly close to IPv6's prefix + interface-ID model and to SLAAC deriving the interface ID from the MAC. In that narrow sense, IPv6 re-learned a good idea XNS/IPX had decades earlier.

Where IPX fell short of being a global internet protocol: it was chatty (constant SAP/RIP broadcasts), LAN-oriented rather than built for global hierarchical routing, and proprietary to Novell. As “the internet” became the thing everyone connected to, IPX faded — Novell eventually bolted IP onto NetWare, which then declined in the late '90s.

WHY IT MATTERS: The irony worth leaving people with: IPv4 won by being open and good-enough, and NAT then kept it good-enough for decades. That very success is exactly why IPv6 — ready in 1995 — took 25+ years to deploy. We managed the scarcity instead of solving it, and managing it was cheap enough to postpone the fix a generation.

2. How IPsec is “intrinsic” to IPv6

You'll hear that IPv6 has “built-in security.” That's half-true and half-myth, and the distinction is worth getting right.

The true part: it's part of the header design

IPsec is two protocols, and in IPv6 both are simply extension headers — they slot into the same next-header chain as the Routing, Fragment, and Hop-by-Hop headers. Security wasn't bolted on after the fact; the packet format was designed with a slot for it.

Piece	Proto #	What it does	Notes
AH — Authentication Header	51	Integrity + authenticity of the WHOLE packet, including the IP header. No encryption.	Breaks through NAT (it authenticates the addresses NAT rewrites).
ESP — Encapsulating Security Payload	50	Encrypts (and optionally authenticates) the payload.	The one actually used in the field; NAT needs NAT-T (UDP 4500).

Originally the IPv6 Node Requirements (RFC 4294, 2006) said a compliant stack **MUST** implement IPsec. That universal-baseline expectation is where “intrinsic” comes from: you could assume any IPv6 node could do it, with no add-on.

Why it genuinely matters: no NAT

This is the part that's easy to miss. AH authenticates the entire packet including source and destination addresses, so any device that rewrites addresses — i.e. NAT — invalidates it. ESP fares a little better but still needs NAT-traversal hacks. Because IPv6 removes NAT from the picture, real end-to-end IPsec becomes practical rather than theoretical.

WHY IT MATTERS: The clean version to say out loud: “IPv6 didn't make IPsec stronger — it made the network flat enough (no NAT) for end-to-end IPsec to actually work, and it gave the protocol a native header slot for it.”

The myth part: it is NOT on by default

COMMON MYTH: “IPv6 traffic is encrypted/secured automatically” — No. “Supported” is not “enabled.” You still need Security Associations, a key-exchange mechanism (IKEv2), and policy before a single packet is protected. And in 2011 RFC 6434 downgraded IPsec from **MUST** to **SHOULD** (RFC 8504 keeps it **SHOULD** today), precisely because constrained/IoT devices couldn't always carry it and because security had moved up the stack.

In real operations, data-in-transit confidentiality is overwhelmingly handled by TLS at the application layer and by service meshes (mTLS) — not by network-layer IPsec. So treat “intrinsic IPsec” as architectural elegance and a consistent baseline, not as a feature you'll rely on daily.

But what about NAT in IPv6? (and does it break AH/ESP?)

NAT didn't actually vanish in IPv6 — it just lost its main job (address conservation) and survives in a few narrower forms. Each has a different relationship with IPsec.

Flavor	What it does	Why it exists
NPTv6 (RFC 6296)	Stateless, 1:1 prefix rewrite (keeps the interface ID)	Multihoming / provider-independence & topology hiding — NOT address saving
Stateful NAT66	Port-overloading v6→v6 (the IPv4 NAPT analog)	Exists (Linux, some vendors) but the IETF actively discourages it
NAT64 (RFC 6146)	Translates v6 ↔ v4 (with DNS64 / 464XLAT)	Lets IPv6-only clients reach the legacy IPv4 internet — huge on mobile

Effect on AH

Broken by all of them, by design. AH authenticates the whole packet including the IP header, so any address or prefix rewrite invalidates the ICV — that's exactly the tampering AH exists to catch. RFC 6296 even spells this out: NPTv6 will not work with AH. (It's a big reason AH is rarely deployed.)

Effect on ESP — and the nice IPv6 twist

Much gentler, because ESP's integrity check doesn't cover the outer IP header, so a prefix rewrite doesn't fail it directly. In IPv4 the real killer was NAPT: ESP is IP protocol 50 with no ports, so a port-overloading NAT can't tell two ESP flows apart — hence NAT-T (RFC 3948), which wraps ESP in UDP/4500.

NPTv6 sidesteps that entirely: it's stateless and 1:1, so there's no port-demux problem, and it is deliberately checksum-neutral — it compensates the prefix change so the ones-complement checksum is unchanged. That means even transport-mode ESP, whose encrypted transport checksum covers the IP pseudo-header and can't be touched in transit, still validates at the receiver. Tunnel-mode ESP was never an issue. So NPTv6 + ESP generally just works; NAT64, being a heavier v6↔v4 translation, normally needs IPsec tunnelled/encapsulated to cross it.

WHY IT MATTERS: The takeaway: AH is the consistent casualty of any address rewrite in either protocol; ESP survives translation far more gracefully under IPv6's NPTv6 (stateless, checksum-neutral) than it ever did behind IPv4 NAPT. NAT in IPv6 is a niche tool, not a way of life.

Read further

- RFC 4301 — Security Architecture for IP (the IPsec framework)
- RFC 4302 (AH) and RFC 4303 (ESP) — the two extension headers themselves
- RFC 7296 — IKEv2 (how keys/SAs are actually negotiated)

- RFC 4294 → RFC 6434 → RFC 8504 — the IPv6 Node Requirements lineage (watch MUST become SHOULD)
- RFC 6296 — NPTv6 · RFC 6146 — stateful NAT64 · RFC 3948 — UDP encapsulation of ESP (NAT-T)

3. Why there's no broadcast in IPv6

IPv6 deliberately has no broadcast. It isn't a missing feature — it's a redesign. The jobs broadcast did in IPv4 are handled by multicast (“talk to exactly the group that cares”) and anycast (“talk to the nearest one”).

Why broadcast was removed

Broadcast is rude and doesn't scale: a broadcast frame forces EVERY host on the segment to interrupt its CPU, inspect the packet, and usually discard it — even though it wasn't meant for them. IPv4 does this constantly (every ARP is a broadcast). IPv6 replaces the blunt instrument with targeted multicast groups, so uninterested hosts filter the traffic out in hardware and never wake the CPU.

WHY IT MATTERS: The flagship example is ARP's replacement. Instead of broadcasting “who has 10.0.0.5?” to the whole subnet, IPv6 Neighbor Discovery sends to the target's solicited-node multicast group — usually only the intended host is listening. Same job, a fraction of the noise.

How IPv6 multicast differs from IPv4 multicast

The biggest difference is role, not syntax. In IPv4 multicast is optional and peripheral; in IPv6 it is mandatory and load-bearing — Neighbor Discovery, router discovery, duplicate-address detection, and DHCPv6 all depend on it, so the protocol literally can't function without it.

Aspect	IPv4	IPv6
Role	Optional; mostly for apps & a few routing protocols	Mandatory & foundational to the protocol
Core ops use it?	No — ARP/DHCP use broadcast	Yes — ND, RA/RS, DAD, DHCPv6
Address range	224.0.0.0/4	ff00::/8
Scope	Faked via TTL or admin range 239.0.0.0/8	Encoded in the address (ff02=link, ff05=site, ff0e=global)
Group membership	IGMP	MLD (Multicast Listener Discovery, carried in ICMPv6)
L2 MAC mapping	01:00:5e + 23 bits	33:33 + low 32 bits
Auto-joined groups	None required	all-nodes + a solicited-node group per address

Well-known multicast groups (the broadcast replacements)

Group	Meaning
ff02::1	All nodes on the link (the closest thing to the old “broadcast”)
ff02::2	All routers on the link (used by Router Solicitation)

ff02::1:2	All DHCPv6 relay agents & servers (link scope)
ff02::1:ffXX:XXXX	Solicited-node group (ARP's replacement — see below)
ff02::5 / ff02::6	OSPFv3 AllSPFRouters / AllDRouters
ff02::fb	Multicast DNS (mDNS)

The clever bit: solicited-node multicast

This has no IPv4 equivalent and it's what makes ND quiet. For every unicast/anycast address an interface holds, it also joins a multicast group built from the address's low-order 24 bits, prepended with ff02::1:ff00:0/104. Neighbor Solicitations target that group, so typically only the one host you're looking for (plus the rare address collision) ever processes the request.

```
# host address:      2001:db8::a:b:c:1234:5678
# take the low 24 bits:      34:5678
# solicited-node group: ff02::1:ff34:5678
```

Two more worth a mention: MLD is simply IPv6's version of IGMP (how a host tells routers which groups it wants), and it rides inside ICMPv6 rather than being its own protocol. And anycast — one address configured on several nodes, with the network delivering to the nearest — is a first-class concept in IPv6 (think DNS root servers or a cloud anycast load-balancer VIP).

ARP vs. Neighbor Discovery (ND), side by side

ND is often introduced as “ARP for IPv6,” but that undersells it. ARP did exactly one thing — map an IP address to a MAC — and IPv4 used separate, unrelated mechanisms for everything else. ND folds address resolution, router/gateway discovery, prefix and parameter discovery, redirects, duplicate-address detection, and reachability tracking into a single protocol that runs over ICMPv6 and uses multicast instead of broadcast.

ND is built from five ICMPv6 message types:

- Router Solicitation (RS, type 133) + Router Advertisement (RA, 134) — gateway & prefix discovery; the RA feeds SLAAC
- Neighbor Solicitation (NS, 135) + Neighbor Advertisement (NA, 136) — address resolution (the ARP analog) and DAD
- Redirect (137) — “there's a better next-hop router, use it instead”

Dimension	ARP (IPv4)	Neighbor Discovery (IPv6)
Runs on	Standalone L2 protocol (EtherType 0x0806)	ICMPv6 (IP protocol 58)
Delivery	Broadcast to the whole segment	Multicast (solicited-node group) — no broadcast
Address resolution	ARP request / reply	Neighbor Solicitation / Advertisement
Gateway discovery	Separate (ICMP Router Discovery or DHCP)	Built in (RS / RA)

Prefix / autoconfig	Not provided — needs DHCP	RA carries prefixes → enables SLAAC
Reachability tracking	None — entries just time out	NUD: a per-neighbor cache state machine
Duplicate-addr detection	Ad hoc (gratuitous ARP)	DAD — standardized (NS to your own address)
Redirect to better router	Separate ICMP Redirect	ND Redirect (type 137)
Spoofing defense	Dynamic ARP Inspection (switch feature)	RA Guard / ND inspection; SEND (RFC 3971, rare)
Neighbor cache	Flat ARP table	States: INCOMPLETE → REACHABLE → STALE → DELAY → PROBE

WHY IT MATTERS: The real upgrade is NUD (Neighbor Unreachability Detection). ARP has no way to notice a neighbor went away — stale entries just sit there until they age out. ND actively tracks each neighbor's reachability, so IPv6 spots a dead gateway or a moved host far faster. There is no ARP equivalent.

COMMON MYTH: Don't oversell ND's security — Plain ND is about as spoofable as ARP — a rogue NA can poison a neighbor cache and a rogue RA can hijack the default route. The defenses just have different names (RA Guard and ND inspection on switches, or SEND end-to-end) and, exactly like Dynamic ARP Inspection, must be deliberately turned on.

Read further

- RFC 4291 — IPv6 Addressing Architecture (multicast address format, scopes, solicited-node)
- RFC 4861 / RFC 4862 — Neighbor Discovery & SLAAC (the main consumers of link-local multicast)
- RFC 826 — ARP (the IPv4 original) · RFC 3971 — SEND (SEcure Neighbor Discovery)
- RFC 3810 / RFC 2710 — MLDv2 / MLD (the IGMP equivalent)

4. Errata: claims worth stating precisely

None of these are wrong enough to mislead a beginner, but a sharp attendee may push on them — so here's the precise version of each.

“IPv6 has mandatory, built-in IPsec”

Precise version: IPsec is natively integrated via extension headers and was once a MUST, but RFC 6434 (2011) relaxed it to SHOULD. It's consistently available, not automatically on. (Full treatment in Section 2.)

“IPv6 subnets are always /64”

Precise version: /64 is the convention for host/LAN subnets because SLAAC uses the lower 64 bits as the interface ID. It is not a universal rule: point-to-point links commonly use /127 (RFC 6164) and loopbacks/single hosts use /128 — exactly like /31 and /32 in IPv4.

“:: is the ‘any’ address”

Precise version: there are two different “anys.” :: is the unspecified address — the socket wildcard you bind() to so a service listens on all addresses. ::/0 is the default route — the all-addresses prefix used in route tables, ACLs, and Security Groups (the v6 equivalent of 0.0.0.0/0, i.e. 0/0). Different layers, both loosely called “any.”

“The IPv6 spec is RFC 8200” (today, yes — but mind the lineage)

IPv6 has been re-issued twice. RFC 1883 (1995) was first; RFC 2460 (1998) replaced it; RFC 8200 (2017) replaced that and made IPv6 a full Internet Standard (STD 86). The core never changed — same 128-bit addresses, same 40-byte header. Notable refinements along the way:

- Minimum link MTU raised from 576 to 1280 octets (1883 → 2460).
- Routing Header Type 0 (RH0) deprecated for security (RFC 5095, reflected in 8200).
- Extension-header handling tightened: intermediate routers don't process extension headers except Hop-by-Hop (and even that is no longer mandatory for every node); headers may only be added by the source, not inserted en route (8200).

“The RIRs ran out of IPv4 on [single date]”

Precise version: the dates on the timeline mark when each registry reached its final-/8 / depletion phase and effectively stopped normal allocation. Small reserved and recovered pools (and waiting lists) continued afterward — for example, RIPE NCC kept allocating from recovered space and made its final /22 allocations on 25 November 2019, years after its September 2012 “last /8” milestone.

5. RFC index (matches the timeline diagram)

A one-stop reference for every RFC named across the deck, workbook, and timeline.

IPv4 life-extenders

RFC	Title	Date
1631	The IP Network Address Translator (NAT)	May 1994
1918	Address Allocation for Private Internets	Feb 1996

IPv6 core protocol

RFC	Title	Date
1883	IPv6 Specification (first edition)	Dec 1995
2460	IPv6 Specification (replaced 1883)	Dec 1998
8200	IPv6 Specification — Internet Standard, STD 86	Jul 2017
4291	IPv6 Addressing Architecture	Feb 2006

Addressing, autoconfig & neighbors

RFC	Title	Date
4193	Unique Local IPv6 Addresses (ULA)	Oct 2005
4861	Neighbor Discovery for IPv6 (ND)	Sep 2007
4862	IPv6 Stateless Address Autoconfiguration (SLAAC)	Sep 2007
3315 / 8415	DHCPv6 (3315 original; 8415 consolidated)	2003 / 2018
3849	IPv6 Address Prefix Reserved for Documentation (2001:db8::/32)	Jul 2004

Subnetting, requirements & security

RFC	Title	Date
6164	Using /127 Prefixes on Inter-Router Links	Apr 2011
6434 / 8504	IPv6 Node Requirements (IPsec MUST → SHOULD)	2011 / 2019
4301	Security Architecture for IP (IPsec)	Dec 2005
4302 / 4303	Authentication Header (AH) / ESP	Dec 2005

7296	Internet Key Exchange v2 (IKEv2)	Oct 2014
5095	Deprecation of Routing Header Type 0	Dec 2007

NOTE: These are the publication dates of the RFCs. The IPv4 exhaustion dates on the timeline are registry operational milestones, not RFCs.